

git & GitHub

Data Science for Economics

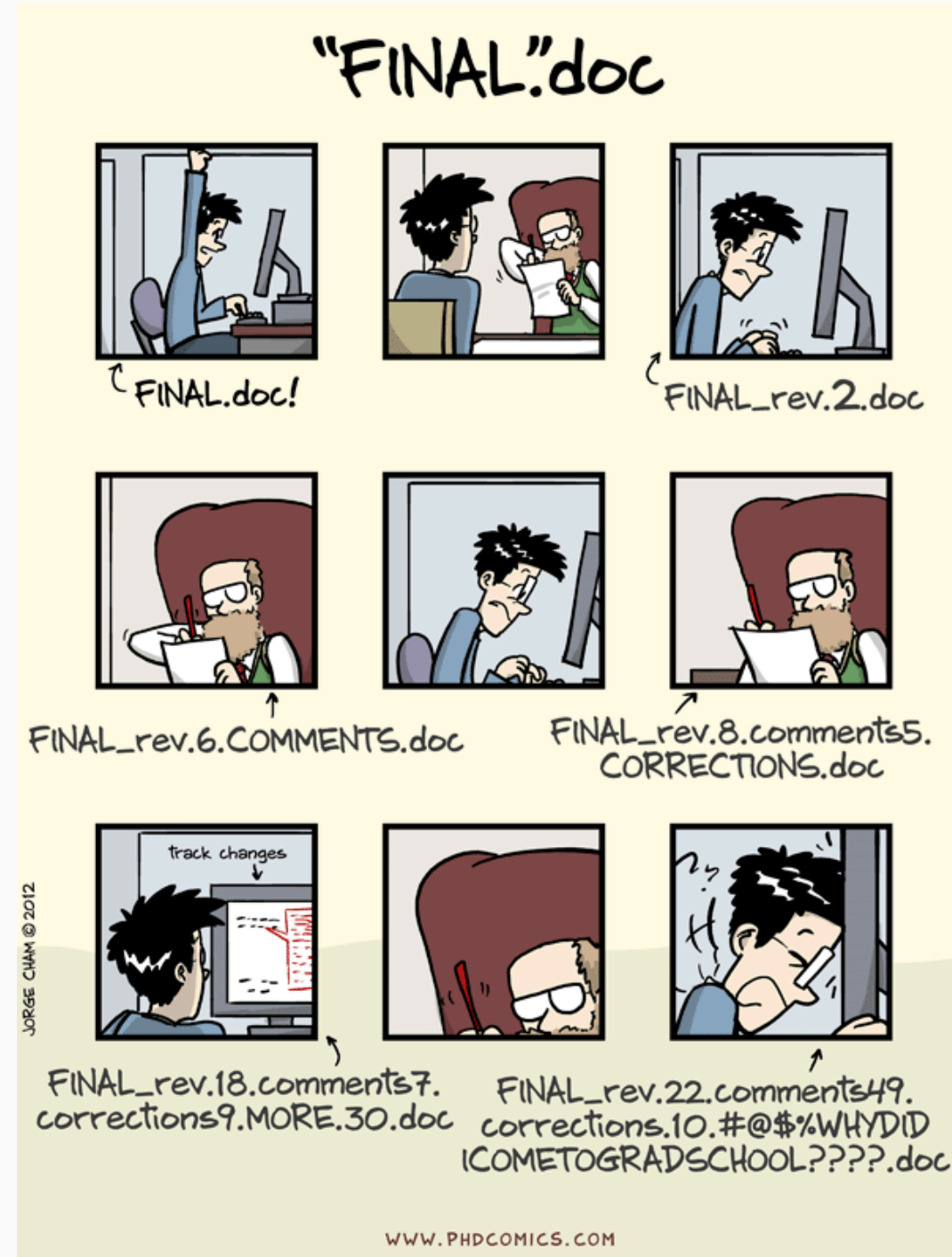
Damian Clarke

dcc213@exeter.ac.uk

Department of Economics, University of Exeter

30 January, 2026

The Problem Version Control Seeks to Resolve 🤯



PhD Comics: "Final".doc

Source: PhD Comics

Why We Need Version Control

The challenges of managing code and documents:

Without version control:

- Multiple versions scattered everywhere
- Unclear which is the “real” version
- Lost work when files get overwritten
- Impossible to track who changed what
- Collaboration nightmares
- Can’t easily revert mistakes

With version control:

- Single authoritative version ✨
- Complete history of all changes
- Easy to see what changed and when
- Collaboration becomes seamless
- Revert to any previous version
- Experiment without fear
- Industry standard for development

Version control isn’t just for code or documents—it’s for an entire project that evolves over time!

What will we do in this unit?

1. Discuss and get to know git
2. Work out how git interacts with github
3. Discuss markdown languages
4. Look at some more advanced themes: how hard is it to build a site together?

Note

Certain elements are covered in this slide, but may be discussed just in passing. Elements marked * are advanced themes.

Let's go :-)

What is git?

git is a distributed version control system created by Linus Torvalds (the Linux creator) in 2005

Key concepts:

- **Repository (repo)**: A project folder tracked by git
- **Commit**: A snapshot of your project at a point in time
- **Branch**: An independent line of development
- **Remote**: A version of your repository hosted elsewhere (e.g., GitHub)

git vs GitHub

git = The version control software (runs on your computer) – you must install

GitHub = A website/service for hosting git repositories (plus collaboration tools) – you must make an account

git's Philosophy 🌳

Distributed version control means:

1. **Everyone has a complete copy** of the project history
2. **Work offline** - you don't need constant internet connection
3. **Fast operations** - most things happen locally
4. **Backup by design** - multiple complete copies exist

The git workflow:

Working Directory → Staging Area → Local Repository → Remote Repository

Key advantage: You can experiment locally without affecting others until you're ready to share!

Installing and Setting Up git

First, check if git is installed:

```
git --version
```

If you see a version number, you're good to go! If not, install from git-scm.com

Initial configuration (do this once):

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"
```

Important!

Use the **same email** you'll use for your GitHub account. This links your commits to your GitHub profile.

Verify your configuration:

```
git config --list
```


Creating Your First Repository 🚀

Two ways to start:

Option 1: Start a new project

```
mkdir myproject  
cd myproject  
git init
```

Option 2: Clone an existing project

```
git clone https://github.com/username/repository
```

What just happened?

`git init` creates a hidden `.git` folder that stores all version history. You won't see it with `ls`, but it's there!

```
ls -a # Shows hidden files including .git
```


The git Workflow: Basic Commands

Essential commands you'll use constantly:

Command	Function	Example
<code>git status</code>	See what's changed	<code>git status</code>
<code>git add</code>	Stage files for commit	<code>git add file.txt</code>
<code>git commit</code>	Save a snapshot	<code>git commit -m "Add feature"</code>
<code>git log</code>	View commit history	<code>git log</code>
<code>git diff</code>	See changes	<code>git diff file.txt</code>

The three states of files:

1. **Modified**: Changed but not staged
2. **Staged**: Marked for inclusion in next commit
3. **Committed**: Safely stored in repository

Your First Commits

Let's walk through a complete example:

```
# Create a file
echo "Hello world!" > file1.txt

# Check status
git status

# Stage the file
git add file1.txt

# Check status again
git status

# Commit it!
git commit -m "Adds hello world file"

# View history
git log
```

Commit Messages

Write clear, descriptive commit messages! Future you (and your collaborators) will thank you.

✓ Good: "Add function to calculate mean"

✗ Bad: "stuff", "changes", "asdf"

Working with Multiple Files

Stage and commit multiple files together:

```
# Make changes to multiple files
echo "Some new output" >> file1.txt
echo "A new file" > file2.txt

# Stage specific files
git add file1.txt file2.txt

# Or stage all changes
git add .

# Commit them together
git commit -m "Add new function with supporting file"
```

In-Class Activity

1. Create a new git repository
2. Add 2-3 files to it
3. Make some changes
4. Practice staging and committing
5. Use `git log` to see your history

Understanding git log

View your commit history:

```
git log
```

Each commit shows:

- **Commit hash:** Unique identifier (long string of letters/numbers)
- **Author:** Who made the commit
- **Date:** When it was committed
- **Message:** What changed

Useful log options:

```
git log --oneline      # Condensed view
git log --graph        # Shows branching
git log --all --decorate # Shows all branches
git log -n 5           # Last 5 commits
git log --author="Name" # Filter by author
```

Viewing Changes with git diff

See what's changed before committing:

```
# See unstaged changes
git diff

# See staged changes
git diff --staged

# Compare specific commits
git diff commit1 commit2

# Compare with a previous version
git diff HEAD~1 # Compare with 1 commit ago
```

Reading diff output

- Lines starting with `-` (red) were removed
- Lines starting with `+` (green) were added
- Lines without symbols are context

Introduction to GitHub

What is GitHub?

GitHub is a cloud-based platform for hosting git repositories with additional features:

Core features:

- Remote repository hosting
- Web-based interface
- Backup and accessibility
- Portfolio of your work

Collaboration features:

- Pull requests for code review
- Issues for tracking bugs/features
- Project management tools
- Team access control
- Documentation (wikis, README)

GitHub is essential for:

- Open source projects
- Team collaboration
- Job applications (your portfolio!)
- Learning from others' code

Creating a GitHub Account

Steps to get started:

1. Go to github.com
2. Sign up with your email (use your university email for student benefits!)
3. Choose a professional username (employers will see this!)
4. Verify your email

Student Benefits

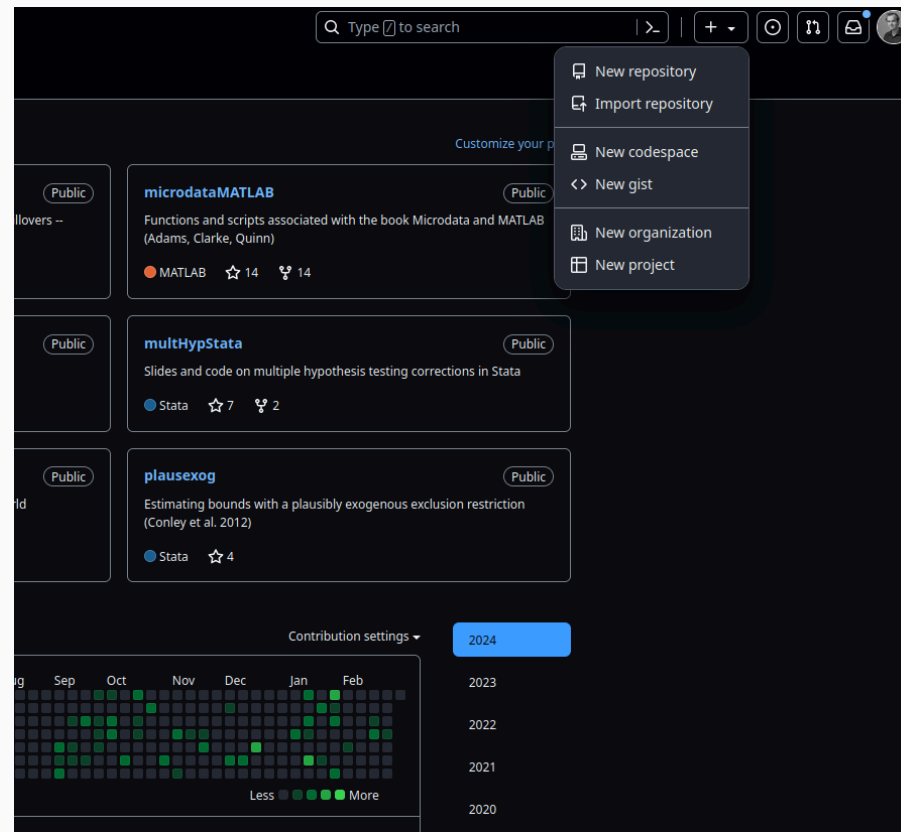
With a university email, you get:

- GitHub Pro for free
- Free private repositories
- Access to GitHub Copilot
- Many other development tools

Apply at: education.github.com

Publishing Your Repository on GitHub

Create a new repository on GitHub:



Creating a new GitHub repository

Then connect your local repository:

```
# Add GitHub as remote
git remote add origin https://github.com/username/repository

# Check it worked
git remote -v

# Push your code
git branch -M main
git push -u origin main
```


Understanding Push and Pull

Two-way synchronization:

Push (): Send local commits to GitHub

```
git push origin main
```

When to push:

- After completing a feature
- At end of working session
- Before asking others to review
- To backup your work

Pull (): Get updates from GitHub

```
git pull origin main
```

When to pull:

- **Always at start of session!**
- Before starting new work
- When collaborators make changes
- To sync across devices

Golden Rule

Always `git pull` **before you start working!** This prevents conflicts with collaborators' changes.

Authentication on GitHub

GitHub requires authentication for pushing code

Options:

1. **Personal Access Token (PAT)** - Recommended
2. SSH keys
3. GitHub CLI

Creating a PAT:

1. GitHub → Settings → Developer settings → Personal access tokens
2. Generate new token (classic)
3. Select scopes (at minimum: `repo`)
4. Copy token (you'll only see it once!)
5. Use token as password when pushing

Storing credentials

After first use, git can remember your credentials. See [GitHub docs](#) for details.

README Files

Every repository should have a README.md

Written in Markdown, it's the first thing people see on GitHub:

Project Title

Brief description of what this project does

Installation

How to install/use

Usage

Examples of how to use

Contributing

How others can contribute

License

What license (if any)

Add and commit your README:

```
git add README.md
git commit -m "Add README"
git push origin main
```

Introduction to Markdown

What is Markdown?

Markdown is a lightweight markup language for creating formatted text using plain text

Why learn Markdown?

It's everywhere:

- GitHub (READMEs, comments, issues)
- Documentation sites
- Jupyter notebooks
- Slack, Discord, WhatsApp
- Static site generators
- This presentation! (via Quarto)

Advantages:

- Easy to read and write
- Plain text (version control friendly!)
- Platform independent
- Quick formatting without mouse
- Converts to HTML, PDF, Word...

Philosophy: Focus on writing, not formatting. Readable even as plain text!

Markdown Basic Syntax

The essentials you'll use constantly:

Element	Markdown	Result
Heading	# H1, ## H2, ### H3	Headers (different sizes)
Bold	**bold text**	bold text
Italic	<i>*italic text*</i>	<i>italic text</i>
Code	<code>`code`</code>	<code>code</code>
Link	[text](url)	text
Image	![alt](image.jpg)	Displays image
List	- item or 1. item	Bullet or numbered list
Quote	> quote	Block quote

Pro Tip

You can nest formatting: *****bold_and_italic***** produces ***bold and italic***

Markdown: Lists and Code Blocks

Unordered lists:

- First item
- Second item
 - Nested item
 - Another nested item
- Third item

Ordered lists:

1. First item
2. Second item
3. Third item

Code blocks:

```
```python
def hello():
 print("Hello, world!")
```
```

Inline code: Use single backticks: ``variable_name``

Markdown in Practice: README Example

A typical repository README:

```
# Project Title

Brief description of what this project does and who it's for.

## Installation

```bash
git clone https://github.com/username/repo
pip install -r requirements.txt
```

## Usage

```python
import mypackage
result = mypackage.do_something()
```

## Features

- Feature 1: Does X
- Feature 2: Does Y
- Feature 3: Does Z

## Contributing

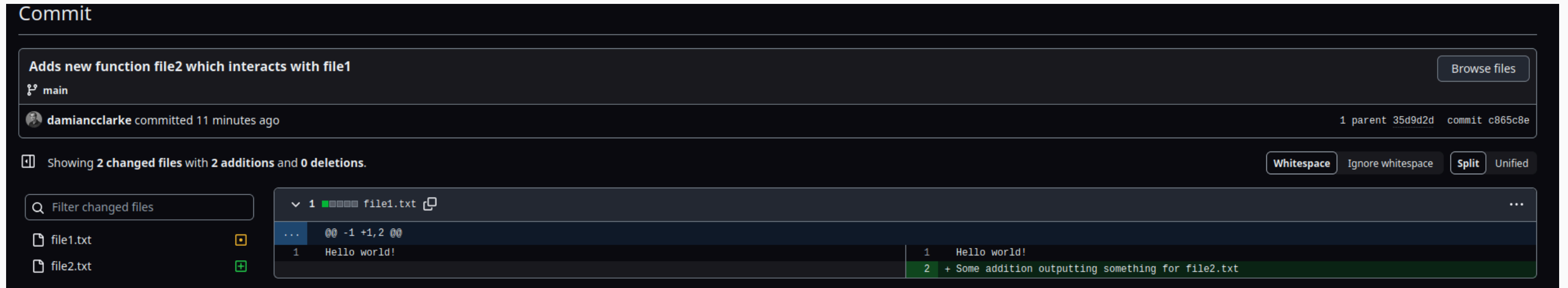
Pull requests are welcome! Please see [CONTRIBUTING.md](CONTRIBUTING.md).

## License
```

This renders very nicely on GitHub (and lots of other places) and is easy to edit!

Visualizing Changes on GitHub

GitHub makes it easy to see what changed:



The screenshot shows a GitHub commit page for a commit titled "Adds new function file2 which interacts with file1". The commit was made by damianclarke 11 minutes ago. It shows 2 changed files: file1.txt and file2.txt. The file1.txt diff shows a line being added (green background) with the text "Hello world!". The file2.txt diff shows a line being added (green background) with the text "+ Some addition outputting something for file2.txt". The interface includes a search bar for changed files, a list of changed files, and a diff view with line-by-line comparison and color coding (red for removed, green for added).

Visualizing commit changes

Features:

- Line-by-line comparison
- Color coding (red = removed, green = added)
- Easy to review others' work
- Link to specific lines in discussions

Forking Repositories

Forking creates your own copy of someone else's repository

Why fork?

- Contribute to open source projects
- Experiment with others' code
- Create your own version of a project
- Learn from existing code

The workflow:

1. Click "Fork" button on GitHub
2. Clone YOUR fork to your machine
3. Make changes locally
4. Push to YOUR fork
5. Open a Pull Request to original repository

Contributing to Others' Projects 🤝

The standard workflow:

```
# 1. Fork the repository on GitHub (click "Fork")

# 2. Clone YOUR fork
git clone https://github.com/YOUR-USERNAME/repository

# 3. Make changes
echo "My contribution" > file3.txt

# 4. Stage and commit
git add file3.txt
git commit -m "Add new feature"

# 5. Push to YOUR fork
git push origin main

# 6. Open Pull Request on GitHub
```

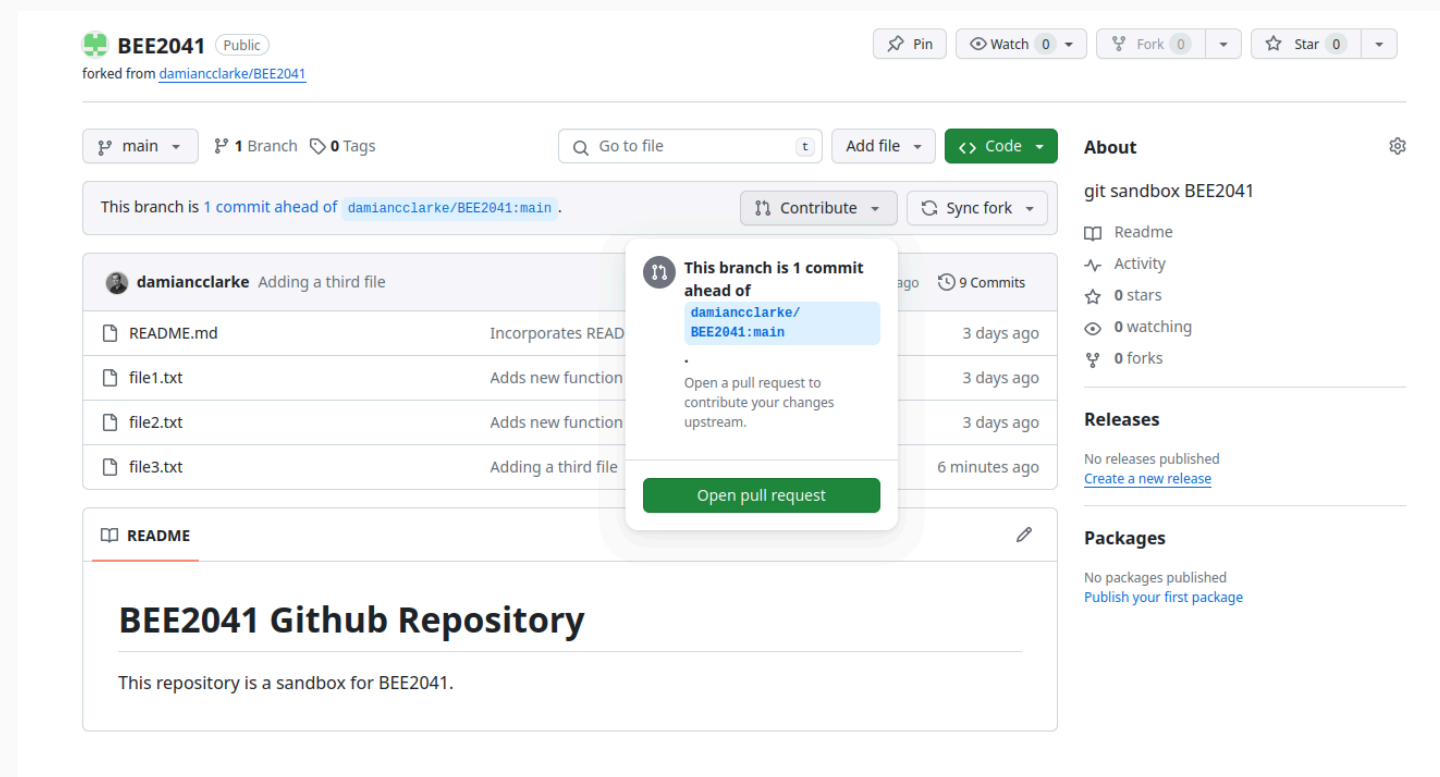
This is how open source development works! ✨

Pull Requests Explained

A Pull Request (PR) asks the maintainer to “pull” your changes

The PR process:

1. **You:** Fork → Clone → Edit → Commit → Push → Create PR
2. **Maintainer:** Reviews your changes
3. **Discussion:** Comments, questions, requests for changes
4. **Decision:** Merge (accept) or Close (reject)



Creating a pull request

Reviewing and Merging Pull Requests

If you maintain a repository, you'll review PRs (we'll see this together)

Review process:

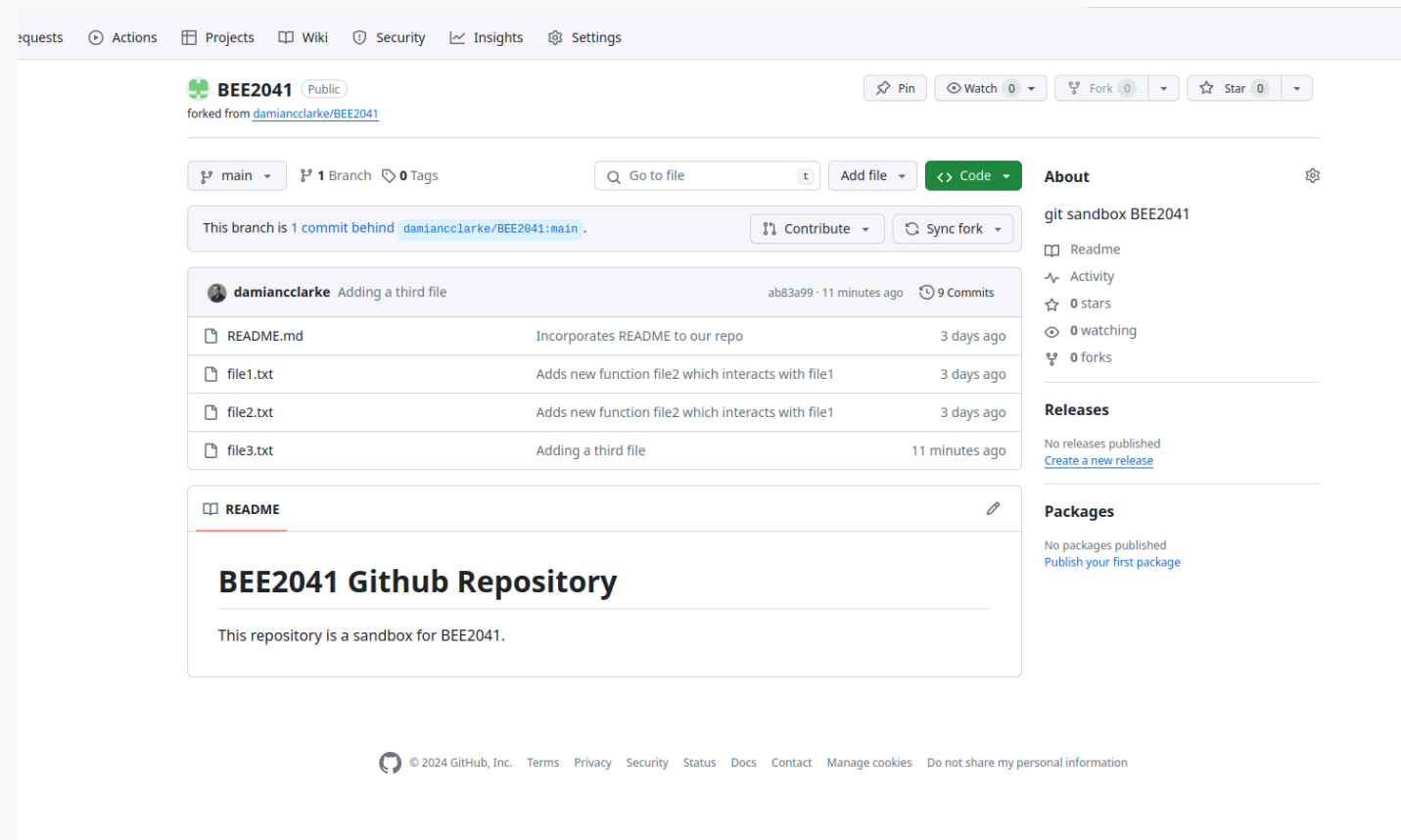
1. Read the description
2. Examine code changes
3. Check for conflicts
4. Test if possible
5. Leave comments or approve
6. Merge or request changes

In-Class Activity

1. Fork the class repository
2. Make a small change (add a file with your name)
3. Commit and push to your fork
4. Open a Pull Request to the original repository

Syncing Your Fork

After the maintainer merges changes, your fork is behind:



Repository behind main

To sync:

On GitHub: Click “Sync fork” button

Locally:

```
git pull origin main
```

This keeps your fork up-to-date with the original repository!

* Branching: Parallel Development

Branches let you work on features without affecting main code

Why use branches?

- Experiment safely
- Work on multiple features simultaneously
- Keep main branch stable
- Easier code review (one feature per PR)

Branch commands:

```
git branch          # List branches
git branch new-feature # Create branch
git checkout new-feature # Switch to branch
git checkout -b new-feature # Create and switch in one command
```

Branch naming

Use descriptive names: `add-login-feature`, `fix-bug-123`, `update-documentation`

* Working with Branches

Typical workflow:

```
# Create and switch to new branch
git checkout -b new-feature

# Make changes
echo "New feature code" > feature.txt
git add feature.txt
git commit -m "Implement new feature"

# Switch back to main
git checkout main

# Merge the feature
git merge new-feature

# Delete the branch (optional)
git branch -d new-feature
```

The branch only exists until you merge it into main!

* Pushing Branches to GitHub

Share your branch on GitHub:

```
# Push branch to GitHub  
git push origin new-feature
```

Benefits:

- Backup of work in progress
- Others can see/review before merging
- Can open PR from the branch
- Collaborate on the same feature

On GitHub: You can open a PR from your branch to main without merging locally first!

* Understanding merge

Merging combines changes from different branches

```
# Switch to main branch
git checkout main

# Merge feature branch
git merge new-feature
```

What happens:

- git combines the changes
- Creates a merge commit (usually)
- Both histories are preserved

Possible outcomes:

- **Fast-forward:** Clean merge, just moves pointer
- **Merge commit:** Creates new commit combining both branches
- **Conflict:** Manual intervention needed (we'll discuss next)

Dealing with Conflicts

Conflicts occur when same lines are edited differently

When you see a conflict:

```
git merge new-feature
# Auto-merging file.txt
# CONFLICT (content): Merge conflict in file.txt
# Automatic merge failed; fix conflicts and then commit the result.
```

The file will look like:

```
<<<<<< HEAD
Your version of the line
=====
Their version of the line
>>>>>> new-feature
```

To resolve:

1. Open the file
2. Decide what to keep
3. Remove the conflict markers (`<<<<<<`, `=====`, `>>>>>>`)
4. Stage and commit

```
git add file.txt
git commit -m "Resolve merge conflict"
```

Best Practices for git & GitHub ✨

Commit practices:

- Commit early and often
- Write meaningful commit messages
- Each commit should be logical unit
- Don't commit broken code to main

Branch practices:

- Keep main branch stable
- Use feature branches
- Delete branches after merging
- Pull main before creating new branch

Collaboration practices:

- Always pull before starting work
- Push regularly (backup!)
- Use pull requests for review
- Respond to code review comments
- Write good documentation (README)

Common git Commands Cheat Sheet

| Category | Command | Purpose |
|---------------|-----------------------------------|-----------------------|
| Setup | <code>git init</code> | Initialize repository |
| | <code>git clone URL</code> | Clone repository |
| Basic | <code>git status</code> | Check status |
| | <code>git add FILE</code> | Stage changes |
| | <code>git commit -m "msg"</code> | Commit changes |
| | <code>git log</code> | View history |
| Remote | <code>git push</code> | Upload to GitHub |
| | <code>git pull</code> | Download from GitHub |
| | <code>git remote -v</code> | View remotes |
| Branch | <code>git branch</code> | List branches |
| | <code>git checkout -b NAME</code> | Create & switch |
| | <code>git merge BRANCH</code> | Merge branch |

Undoing Changes

Different ways to undo depending on stage:

Before staging:

```
git restore file.txt # Discard changes
```

After staging but before commit:

```
git restore --staged file.txt # Unstage
```

After commit:

```
git revert HEAD # Create new commit that undoes last commit
```

Nuclear option (dangerous!):

```
git reset --hard HEAD # Discard all changes since last commit
```

 **Be Careful!**

Some operations (like `reset --hard`) are destructive and can't be undone!

.gitignore Files 🙈

Tell git to ignore certain files:

Create a `.gitignore` file in your repository:

```
# Ignore compiled files
*.pyc
*.o

# Ignore data files
*.csv
data/

# Ignore environment files
.env
venv/

# Ignore OS files
.DS_Store
Thumbs.db

# Ignore IDE files
.vscode/
.idea/
```

Why ignore files?

- Large data files (use Git LFS instead)
- Sensitive information (passwords, API keys) or personal settings
- Generated/temporary files

Real-World Workflows

Individual researcher:

```
git pull                # Get latest
# Work, make changes
git add .
git commit -m "Analysis update"
git push                # Backup
```

Team project:

```
git pull                # Get teammates' changes
git checkout -b my-feature # New feature branch
# Work on feature
git add .
git commit -m "Add feature"
git push origin my-feature
# Open PR on GitHub for review
# After approval, merge to main
```


GitHub Features Beyond git

Issues: Track bugs and feature requests

Projects: Organize work with kanban boards

Actions: Automated workflows (testing, deployment)

Pages: Host websites directly from repository

Wikis: Project documentation

Releases: Tag and package versions

Discussions: Community forum

These make GitHub more than just git hosting: it's actually a complete development platform!

Resources and Further Learning

Official documentation:

- [git documentation](#)
- [GitHub documentation](#)

Interactive tutorials:

- [Learn Git Branching](#)
- [GitHub Skills](#)

Books:

- “Pro Git” by Scott Chacon (free online)
- [git book](#)

Cheat sheets:

- [GitHub git cheat sheet](#)

Practice Exercises 🏆

This week, please complete:

1. **Create a GitHub account** if you don't have one
2. **Create a new repository** on GitHub with README and license
3. **Clone it locally** and add some files
4. **Practice the basic workflow:** add, commit, push
5. **Fork the class repository** and make a contribution (add a file with your name)
6. **Open a Pull Request** to the class repository
7. **Create a branch**, make changes, and merge it
8. **Explore some popular repositories** on GitHub to see real-world examples

Common Mistakes to Avoid

Don't:

- Commit large binary files
- Commit sensitive data (passwords!)
- Work directly on main in team projects
- Force push to shared branches
- Commit generated files (builds, logs)
- Use git as a backup system for data
- Forget to pull before working

Do:

- Write clear commit messages
- Commit logical chunks of work
- Pull before starting work
- Push regularly
- Use .gitignore appropriately
- Keep commits focused
- Review changes before committing

Remember

Git can't protect you from everything! Always be careful with:

- `git push --force`
- `git reset --hard`
- `git clean -fd`

git in Your Research Workflow

How economists and data scientists use git:

For code:

- Track analysis scripts
- Version control for packages
- Collaborate on research code
- Share replication files

For data (carefully!):

- Document data cleaning
- Track data transformations
- Version small datasets
- Use Git LFS for larger files

For papers:

- Version LaTeX files
- Track co-author changes
- Revert to previous versions
- See who wrote what

Key Takeaways

1. **Version control is essential** for any project that evolves over time
2. **git provides powerful local version control** with complete history
3. **GitHub enables collaboration** and serves as backup/portfolio
4. **The basic workflow** is: pull → edit → add → commit → push
5. **Branches allow experimentation** without breaking main code
6. **Pull Requests enable code review** before merging
7. **Practice makes perfect** - start using git for all your projects!

git seems complicated at first, but becomes second nature with practice! 💪

Next Steps

Additional points to explore:

- Deploying sites
- GitHub Actions 
- Other tools which may be of interest (Jekyll, Quarto, D3.js, plotly)

For now:

- Get comfortable with basic commands
- Practice on your own projects
- Explore GitHub repositories
- Start building your portfolio!

Remember: Every expert was once a beginner. Start small, practice often! 